RAG Assignment: Baseline vs. Re-Ranking

Name: Satyajit Priyadarshi Roll No: 2025201055
GitHub: https://github.com/SATYAJIT2747/LMA-RAG-Assignment-1
Document used: *D&D Basic Rules (2018, v1.0)*, 180 pages

0. What I built (short setup)

- **Document:** D&D Basic Rules. I picked it because it has many general rules plus many small exceptions (race traits, conditions, spell restrictions) spread over different chapters. That is exactly where retrieval can go wrong.
- **Chunking:** removed the page header/footer text, then split each page into ~800 character chunks with 100 overlap. Total = **1113 chunks**. Both pipelines use the same chunks.
- **Bi-encoder:** `all-MiniLM-L6-v2` + FAISS `IndexFlatIP` on normalized vectors (= cosine similarity).
- **Cross-encoder:** `cross-encoder/ms-marco-MiniLM-L-6-v2`.
- **LLM:** `gemini-flash-lite-latest`, temperature 0. Same prompt for both answers, it says "answer only from context".
- **Pipeline 1:** Top 10 by cosine → first 3 go to the LLM (Answer 1).
  Pipeline 2: same Top 10 → cross-encoder scores all 10 → sort → top 3 go to the LLM (Answer 2).
- To measure rank shifts properly, for every test query I wrote down the exact sentence from the rulebook that answers it (the "gold" chunk), and the code finds its rank before and after re-ranking.

1. Architecture Comparison

Bi-encoder: the query and each chunk are passed through the model *separately*. Each one becomes a single vector, and we compare them with cosine similarity. Because the chunk vectors don't depend on the query, I can compute all 1113 of them once and store them in FAISS. At query time only the query is encoded, so search is very fast. The downside is that the whole meaning of a chunk is squeezed into one 384-number vector before the model ever sees the question. Small but important words like "not", "except" or "dwarf" can get lost, and two passages that are just about the same topic look similar even if one doesn't answer the question.

Cross-encoder: the query and the chunk are joined into one input (`[CLS] query [SEP] chunk`) and go through the transformer *together*. Self-attention lets every word of the question look at every word of the chunk, so the model can check things like "is this rule about dwarves?" or "does this sentence say the thing is *not* allowed?". It outputs one relevance score instead of a vector. It is much more accurate, but nothing can be precomputed: every (query, chunk) pair needs its own full forward pass.

In short: bi-encoder = fast, good for recall over the whole corpus; cross-encoder = slow, good for precision on a short list. That's why we use them in two stages.

2. Qualitative & Rank Shift Analysis

2.1 Rank changes of the gold chunk

#	Failure type	Query	Gold chunk: baseline rank → re-ranked rank	Gold in top 3? (P1 / P2)
1	Exception / corner case	*Can a dwarf's speed be reduced by heavy armor?*	Chunk 92 (p.15, Dwarf traits): **#4 → #1**	No / Yes
2	Negation & constraints	*Is a creature that is unconscious automatically failing Strength saving throws?*	Chunk 1082 (p.171, Unconscious): **#10 → #1** Chunk 1089 (p.172): **#5 → #2**	No / Yes
3	Keyword trap / distractor	*What gives a spellcaster's healing magic no effect?*	Chunk 709 (p.102, Prayer of Healing): **#8 → #1** Chunk 695 (p.100): **#6 → #2** Chunk 676 (p.97, Heal): #3 → #3	Partly / Yes
4	Buried at rank 4–10 (negation)	*Which creatures does cure wounds not work on?*	Chunk 622 (p.90, Cure Wounds): **#4 → #2**	No / Yes
5	Buried at rank 4–10 (partial)	*Do I suffer any drawback for using a bow against an enemy standing right beside me?*	Chunk 535 (p.76, Ranged attacks in close combat): #8 → #4	No / No
6	Limitation	*How do I knock an enemy out instead of killing them?*	Not in baseline Top 10 at all	No / No

2.2 Answer 1 vs. Answer 2

Test 1 – Dwarf + heavy armor (exception). The baseline top 3 were all general armor chunks (armor proficiency, carrying capacity, heavy armor Strength requirement), because "heavy armor" and "speed" match those strongly. The dwarf trait chunk is mostly about age, size and ability scores, so its single vector was less similar.

Answer 1 stated the general rule (speed −10 ft if Strength is too low) and said the context does not mention dwarves → it **misses the exception**, and a player would get the wrong rule.

Answer 2 correctly said a dwarf's speed is **not** reduced by heavy armor, and also mentioned the general rule for others.

Test 2 – Unconscious creature (negation/constraint). The bi-encoder liked chunks about "falling unconscious at 0 HP" and "death saving throws", since they share the words unconscious + saving throw. The actual rule is in Appendix A (Conditions), a bullet list, and was at rank 5 and 10.

Answer 1: "the context does not contain the answer" → **essential clause omitted**.

Answer 2: "Yes, it automatically fails Strength and Dexterity saving throws" → correct.

Test 3 – Healing magic "no effect" (keyword trap). The ranks 1–2 in the baseline were the general "Magic" intro and the "Healing" section, high word overlap ("magic", "healing", "spellcaster") but they don't say when healing fails. The real answer uses different words: "no effect on *undead or constructs*".

Answer 1 only found it for the *Heal* spell (chunk at #3) and said nothing else is mentioned → **incomplete**.
Answer 2 had three healing spells in context (Prayer of Healing, the p.100 spell and Heal) and gave the general restriction "no effect on undead or constructs" for all of them → more complete.

Test 4 – Cure Wounds restriction. The baseline put the general "Healing" and "stable creature" chunks first and even a monster stat block (Bite attack) at #3. The Cure Wounds spell itself was #4.
Answer 1: "no mention of which creatures cure wounds does not work on" → failed.
Answer 2: "no effect on undead or constructs" → correct.

Test 5 – Bow next to an enemy (partial). The cross-encoder moved the right chunk from #8 to #4, but it still did not reach the top 3.
Answer 1 was the worst case: it took a rule about weapon properties and **wrongly said bows always get disadvantage**, which is a misstated rule / hallucination.
Answer 2 said the context does not contain the answer. That is still not helpful, but at least it is not wrong.

Test 6 – Knock out (limitation). The correct chunk was not in the Top 10 at all, so the cross-encoder had nothing to promote. Both answers said "not in context". This shows that re-ranking **cannot fix recall**: if stage 1 misses it, stage 2 can't recover it.

2.3 Small overall evaluation (20 queries, retrieval only)

Metric	Pipeline 1 (baseline)	Pipeline 2 (re-rank)
Hit@3 (gold chunk in context)	0.55	0.65
MRR@10	0.44	0.62
Recall@10 (same for both)	0.95	0.95

Out of 20 queries: 13 improved, 2 stayed the same, 1 was not in Top 10, and **4 got worse** (for example, "How long does it take to recover from exhaustion?" went #1 → #3, and "go without sleep" went #4 → #10). So the cross-encoder is not perfect. The MS MARCO model was trained on web search questions, and short table-like chunks or game-specific wording can confuse it. But on average it clearly puts the right chunk higher.

3. Latency Profiling

Measured on GPU (CUDA), 60 timed runs per stage after a warm-up. LLM time is the Gemini API call (network).

Stage	Pipeline 1: Baseline (ms)	Pipeline 2: Re-rank (ms)
Query embedding + FAISS Top 10	14.7 (p50 14.0)	14.7 (p50 14.0)
Cross-encoder on 10 pairs	–	26.0 (p50 26.0)
Retrieval total	**14.7**	**40.7**
LLM generation (Gemini)	1270 (p50 1131)	1090 (p50 1022)
End-to-end (mean)	**≈ 1285**	**≈ 1131**

Observations:

- Re-ranking adds about **26 ms** per query, so retrieval becomes ~2.8× slower (14.7 → 40.7 ms).
- But the LLM call takes over 1 second, so the extra 26 ms is only about 2% of the full time. In the end-to-end numbers the re-rank pipeline even came out faster, which is just network noise from the API, not because of re-ranking.
- So for this document, re-ranking is basically "free" in terms of what the user notices.
- One-time indexing of all 1113 chunks with the bi-encoder took ~3 s.

4. Scaling Question

Why not run the cross-encoder on the whole collection from the start?

Because the cross-encoder needs the query *and* the chunk together, nothing can be prepared in advance. For every new question you have to run the full transformer once for **every chunk**, so the cost grows linearly with the corpus size, on every single query. A bi-encoder pays that cost only once (offline indexing); at query time it encodes one short query and does a vector search, which FAISS / ANN indexes make very fast even for millions of vectors.

I tested this directly: running the cross-encoder over all 1113 chunks for one query took **~2.3 s** (about 2.1 ms per pair on GPU), compared to **~12 ms** for the bi-encoder search. Extrapolating:

Corpus size (chunks)	Cross-encoder on every chunk (per query)	Cross-encoder on Top 10 only
1,113 (this PDF)	2.3 s	~26 ms
10,000	21 s	~26 ms
100,000	3.5 min	~26 ms
1,000,000	35 min	~26 ms

Interestingly, for this one query the full-corpus cross-encoder also found the dwarf chunk as #1, so quality is not the problem, **cost** is. The two-stage design gets the best of both: the bi-encoder cheaply narrows millions of chunks down to ~10 likely candidates (high recall), and the cross-encoder spends its expensive computation only on those 10 (high precision). The re-rank cost stays constant no matter how big the collection gets. The trade-off, seen in Test 6, is that if the bi-encoder misses the right chunk in stage 1, the re-ranker can never bring it back, so in practice you would retrieve a bit more than 10 (e.g. 20–50) or add keyword search (BM25) to improve recall.

5. Conclusion

On the same chunks and the same prompt, adding a cross-encoder re-ranker fixed 4 of my 6 failure queries. These were the ones about exceptions ("dwarves are not slowed"), negations ("automatically fails", "has no effect on") and keyword traps, where the baseline LLM either missed the key clause or said "not in context". Hit@3 went from 0.55 to 0.65 and MRR from 0.44 to 0.62, for about 26 ms extra per query. Re-ranking can't fix chunks that stage 1 never retrieved, and it sometimes made the ranking worse, so it improves precision but does not replace good first-stage recall.